



Carrera **Desarrollo y Diseño Web**

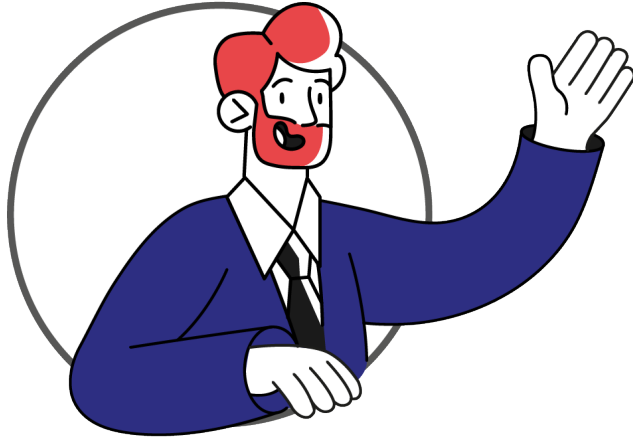
Experiencia 3 – Semana 7

**Conociendo un framework de
JavaScript: jQuery**

Índice

Introducción a la semana.....	3
Resultado de aprendizaje.....	4
Conceptos relevantes.....	6
Preguntas activadoras.....	6
Actividad.....	7
¿Qué es JQuery?.....	8
Vinculación de jquery a un documento HTML.....	9
Uso de jquery.....	11
Selección de elementos del DOM con jquery.....	12
Manejo de eventos en jquery.....	14
Manipulación de elementos en jquery.....	15
Funciones.....	21
Bucles.....	22
De JavaScript a jquery.....	24
Ejemplo de conversión de una página web que utiliza JavaScript a una que utiliza jquery.....	27
Cierre de la semana.....	31
Referencias.....	32
Apuntes.....	33

Introducción a la semana



Las bibliotecas de JavaScript son herramientas que proporcionan soluciones predefinidas y eficientes para tareas comunes, acelerando el proceso de desarrollo. Una de sus ventajas es su sintaxis concisa y su capacidad para abordar las incompatibilidades entre navegadores lo convierten en una herramienta muy útil a la hora de programar las interacciones de tus sitios web. Esta

semana exploraremos una de ellas: jQuery, sus equivalencias con JavaScript nativo, y cómo migrar nuestros programas hechos en las semanas anteriores a esta biblioteca.

Resultado de aprendizaje

El estudiante será capaz de:

RA1. Utiliza elementos básicos de programación en JavaScript en sitio web, de acuerdo criterios de usabilidad y accesibilidad y los requerimientos del proyecto.

RA2. Programa en JavaScript, considerando el control de elementos BOM (browser) y DOM (documento) para un sitio web bajo criterios de usabilidad, accesibilidad y estéticos.

RA3. Utiliza librerías y framework de JavaScript, de acuerdo a requerimientos del diseño del proyecto, para optimizar el proceso de programación de interacción o efectos para el desarrollo de la interfaz.

Indicadores de logro:

IL1. Utiliza correctamente los elementos básicos de programación como variables, arrays, operadores (if - else) y bucles, a fin de dar solución a los requerimientos del proyecto.

IL2. Utiliza el lenguaje JavaScript para programar funciones que permiten reutilizar código de manera eficiente.

IL3. Distingue las funciones y aplicaciones del BOM y DOM en el proceso de desarrollo de un sitio web con JavaScript.

IL4. Utiliza JavaScript para seleccionar y controlar elementos del DOM de manera dinámica, para crear un sitio web que responda a las acciones del usuario.

IL5. Manipula los elementos del DOM, a fin de mejorar la usabilidad, accesibilidad y estética del sitio web.

IL6. Integra la selección, manipulación y control del DOM con el uso de matrices, bucles y funciones.

IL7. Vincula librerías y frameworks de JavaScript en el desarrollo de un sitio web, a fin de crear interfaces interactivas y atractivas de manera más efectiva y eficiente.

IL8. Utiliza frameworks de JavaScript en un proyecto digital, considerando los requerimientos de diseño y las pautas establecidas en la documentación oficial.

Conceptos relevantes

	Biblioteca	JavaScript nativo	Lógica
	jQuery		

Preguntas activadoras

- ¿Te gustaría simplificar la manipulación del DOM y la interacción con elementos de la página web para desarrollar aplicaciones web de manera más eficiente?
- ¿Te interesa acelerar el proceso de desarrollo y mejorar la legibilidad de tu código al utilizar una biblioteca reconocida como jQuery?
- ¿Quisieras explorar una herramienta que te permita manejar fácilmente eventos, animaciones y operaciones comunes en JavaScript sin tener que escribir grandes cantidades de código?

Actividad

Descripción de la actividad

En la séptima semana realizarás una actividad formativa en la que tendrás que convertir un código JavaScript a jQuery y, con ello, replicar el comportamiento que implementaste en JavaScript nativo. Para ello, te basarás en la actividad de la semana 5, retomando las funciones de JavaScript nativo para traspasarlas a JQuery. De esta manera, verás que es posible lograr los mismos resultados mediante planteamientos diferentes y aprenderás a simplificar la manipulación del DOM y otras operaciones comunes mediante jQuery.



Importante

JavaScript nativo se refiere a un conjunto de características y funcionalidades que están integradas directamente en el lenguaje de programación JavaScript, sin depender de bibliotecas externas o frameworks.

¿Qué es JQuery?

jQuery es una biblioteca de JavaScript rápida, liviana y versátil diseñada para **simplificar** la manipulación del DOM (Document Object Model) y el manejo de eventos en páginas web. Su objetivo principal es facilitar la escritura de código JavaScript al proporcionar una interfaz sencilla y consistente para realizar tareas comunes de manera más eficiente. Aunque muchas veces se piensa que JavaScript y jQuery son dos lenguajes de programación diferentes, es importante recordar que jQuery es una biblioteca de JavaScript y no un lenguaje en sí mismo.



Importante

- **Biblioteca:** conjunto de funciones, métodos y utilidades predefinidos que facilitan el desarrollo de aplicaciones web al proporcionar abstracciones y funcionalidades comunes. Estas bibliotecas son escritas en JavaScript y permiten a los desarrolladores realizar tareas específicas de manera más eficiente, reduciendo la cantidad de código que necesitan escribir. Ejemplos de bibliotecas de JavaScript incluyen jQuery, React, Vue.js, y lodash.
- **Lógica:** se refiere a la estructura y secuencia de instrucciones que determinan cómo se deben realizar las operaciones y tomar decisiones dentro de un programa. La lógica de un programa describe el flujo de control, las condiciones y las acciones que deben ejecutarse en respuesta a diferentes situaciones o eventos.

Vinculación de jQuery a un documento HTML

Para utilizar jQuery, primero debes incluir la biblioteca en tu documento HTML. Puedes hacer esto enlazando a la biblioteca a través de una CDN (Content Delivery Network) o descargándola y enlazándola localmente.

La figura 1 muestra un fragmento de código HTML que ejemplifica cómo vincular la biblioteca jQuery en una página web utilizando una CDN. Dentro de la sección <head> del documento HTML, se incluye una etiqueta <script> con un atributo src, el que especifica la URL de la biblioteca jQuery alojada en la CDN.

Figura 1

Ejemplo de enlace a jQuery desde CDN

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <title>Mi Página con jQuery</title>
  <script src="https://code.jquery.com/jquery-3.6.4.min.js"></script>
</head>

<body>
  <!-- Contenido de la página -->
  <script>
    // Tu código jQuery aquí
  </script>
</body>

</html>
```

Links de interés

Para encontrar enlaces a la biblioteca de jQuery, puedes ingresar a los siguientes sitios:

- jQuery: <http://code.jquery.com/>

Este es el sitio oficial de jQuery, donde puedes encontrar las versiones más recientes y estables de la biblioteca jQuery. También encontrarás enlaces a CDN (redes de entrega de contenido), lo que permite una carga más rápida de la biblioteca en tus proyectos web.

- Microsoft:

https://docs.microsoft.com/en-us/aspnet/ajax/cdn/overview#jQuery_Releases_on_the_CDN_0

Esta página forma parte de la documentación oficial de Microsoft y proporciona información sobre las versiones de jQuery disponibles. Es una fuente confiable para obtener jQuery, especialmente, cuando se trabaja en entornos de desarrollo Microsoft.

- Cloudflare: <https://cdnjs.com/>

Cloudflare CDN es un servicio popular que aloja una gran cantidad de bibliotecas de JavaScript, incluyendo jQuery. Ofrece un acceso rápido y confiable a las bibliotecas, optimizando el rendimiento de los sitios web que las utilizan.

- JsDelivr: <https://www.jsdelivr.com/package/npm/jquery>

JsDelivr es un CDN gratuito y de código abierto que proporciona alojamiento para jQuery. Es una excelente opción para implementar jQuery en tus proyectos con alta disponibilidad y rendimiento.

- Google: <https://developers.google.com/speed/libraries#jQuery>

Esta página es parte de la iniciativa de Google para mejorar la velocidad de la web, ofreciendo un CDN para bibliotecas populares como jQuery.

Uso de jQuery

A diferencia de lo que ocurre en JavaScript, donde se incluye el código directamente en los atributos de las etiquetas HTML para interactuar con el DOM (como onclick, onload, etc.), al utilizar jQuery no pondremos el código dentro de las tags de HTML; en su lugar, se generan “observadores” a los objetos del DOM. Por otra parte, al igual que con JavaScript, se coloca al final del código, justo antes del cierre de la etiqueta BODY (</body>), para asegurarnos que los elementos que jQuery buscará o manipulará ya estén cargados.



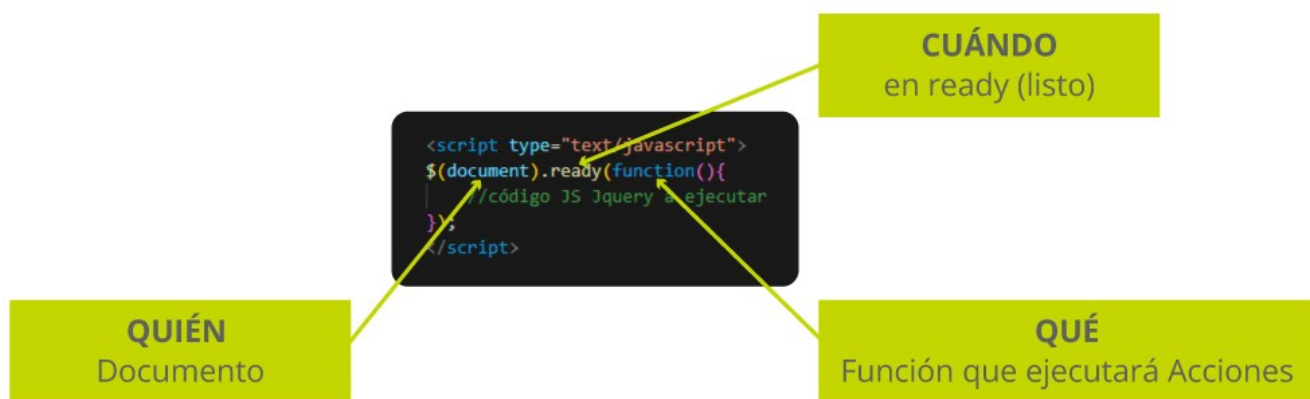
La **sintaxis de jQuery** es una estructura que define primero un **QUIÉN** (objeto o elemento a seleccionar), luego un **CUÁNDO** (evento que desencadenará la acción), y finalmente un **QUÉ** (la o las acciones a realizar). Estructurar el código de esta manera no solo mejora su legibilidad y organización, sino que también puede acelerar el desarrollo al reducir la cantidad de código que se necesita para realizar funciones comunes.

En el ejemplo de la figura 2, se define que el objeto es el documento en sí. Además, se indica que el código se ejecutará una vez que la página web haya sido cargada completamente y se establece una función anónima que se ejecutará una vez que el evento se dispare es anónima.

Es una buena práctica incrustar tus funciones dentro del código mostrado en la figura 2.

Figura 2

Ejemplo de sintaxis de jquery



Selección de elementos del DOM con jQuery

En jQuery, la selección de elementos del DOM se realiza de manera similar a como se hace en JavaScript puro, pero con una sintaxis más concisa: se utiliza el signo \$, seguido de paréntesis y el selector CSS correspondiente. Por ejemplo, \$('<div>.clase') o \$('<div>#id'). Para cada método de selección de elementos nativo de JavaScript, hay un equivalente en jQuery que realiza la misma acción, pero con una sintaxis más simple y corta.

Las equivalencias de los selectores comparados con JavaScript, son las siguientes:

Tabla 1

Equivalencias entre las selecciones de elementos en JavaScript y jQuery

Selección de elementos	
JavaScript	jQuery
document.getElementById('id')	\$('#id')
document.getElementsByClassName('clase')	\$('.clase')
document.getElementsByTagName('tag')	\$('tag')
document.querySelector('div > :first-child')	\$(div > :first-child)
document.querySelectorAll('selector')	\$(selector) o \$(context).find('selector')
document.querySelector('[atributo]')	\$("[atributo]')
document.querySelector('[atributo="valorAtributo"]')	\$("[atributo=valorAtributo]')

Veamos un ejemplo

En la figura 3, se muestra la diferencia entre cómo se seleccionan los elementos todos los elementos de tipo <p> en JavaScript y cómo se realizaría la misma selección utilizando jQuery. En el primer caso, con JavaScript puro, se utiliza la función `getElementsByTagName` para seleccionar todos los elementos <p> del DOM. En el segundo, se muestra el equivalente en jQuery que es más conciso. Aquí, la variable `parrafosjQuery` se asigna con el resultado de la selección jQuery `$('p')`, que también selecciona todos los elementos <p> del DOM.

Figura 3

Comparación de selección de JavaScript y jQuery

```
// JavaScript puro
let parrafosJS = document.getElementsByTagName('p');

// jQuery equivalente
let parrafosjQuery = $('p');
```

Manejo de eventos en jQuery

El manejo de eventos es una parte fundamental en el desarrollo web, ya que permite que las interacciones del usuario con una página o aplicación web desencadenen respuestas específicas. En jQuery, esto es más sencillo y consistente que en JavaScript puro. Puedes usar el método `on()` para asignar event listeners a elementos seleccionados.

Las equivalencias del manejo de eventos, comparados con JavaScript, son las siguientes:

Tabla 2

Equivalencias entre JS y jQuery en el manejo de eventos

Manejo de Eventos	
JavaScript	jQuery
<code>element.addEventListener('evento', callback)</code>	<code>element.on('evento', callback)</code>
<code>element.removeEventListener('evento', callback)</code>	<code>element.off('evento', callback)</code>

Figura 4

Comparación de manejo de eventos JavaScript y jQuery

```
// JavaScript puro
let botonJS = document.getElementById('miBoton');
botonJS.addEventListener('click', function() {
    alert('¡Botón presionado!');
});

// jQuery equivalente
let botonjQuery = $('#miBoton');
botonjQuery.on('click', function() {
    alert('¡Botón presionado!');
});
```

Link de interés

Para un listado completo de eventos que puedes escuchar, visita el siguiente enlace.

<https://api.jquery.com/category/events/>

Manipulación de elementos en jQuery

jQuery proporciona métodos para realizar acciones comunes de manera más sencilla, simplificando la manipulación de elementos en una página o sitio web.

Por ejemplo, para ocultar un elemento en JavaScript, primero se selecciona un elemento por su ID usando `document.getElementById`. Luego, para ocultarlo, se establece la propiedad de estilo `display` del elemento a `'none'`, lo que lo hace invisible en la página.

Con jQuery, el proceso es más sencillo. Primero se selecciona el elemento usando el selector de jQuery `$('#miElemento')`, que es más conciso. Luego, se llama al método `.hide()`, que automáticamente establece la propiedad `display` del elemento a `'none'`, ocultándolo de la vista.

Figura 5

Comparación de acción de ocultar en JavaScript y en jQuery

```
// JavaScript puro
let elementoJS = document.getElementById('miElemento');
elementoJS.style.display = 'none';

// jQuery equivalente
let elementojQuery = $('#miElemento');
elementojQuery.hide();
```

A continuación, revisemos sus equivalencias comparados con JavaScript:

Tabla 3

Equivalencias entre JS y jQuery para manipular elementos

Tipo	Acciones		Qué hace
	JavaScript	jQuery	
Manipulación de Clases	element.classList.add('clase')	\$ (element).addClass('clase')	Agrega una clase de CSS
	element.classList.remove('clase')	\$ (element).removeClass('clase')	Quita una clase de CSS
	element.classList.toggle('clase')	\$ (element).toggleClass('clase')	Si tiene una clase, la quita. Si no la tiene, la agrega. El nombre de la clase es el definido entre paréntesis.
Manipulación de Contenido	element.innerHTML	\$(element).html()	Devuelve el contenido HTML del objeto.
	element.textContent	\$(element).text()	Devuelve el texto (sin HTML) del objeto.
	document.getElementById('miElemento').value (para input y textarea)	\$(element).val()	Devuelve el valor del control.
	var select = document.getElementById('miSelect'); var valorSeleccionado = select.options[select.selectedIndex].value;	\$(element).val()	Devuelve el valor seleccionado del control.
Manipulación de Atributos	element.getAttribute('atributo')	\$(element).attr('atributo')	Obtiene el valor del atributo definido entre paréntesis.
	element.setAttribute('atributo',	\$(element).attr('atributo',	Establece el

	'valor')	'valor')	valor del atributo.
Manipulación de Estilos	element.style.property = 'valor'	\$(element).css('property', 'valor')	Establece el valor de una propiedad css. Por ejemplo, color: blue
Event Listeners	element.addEventListener('evento', callback)	\$(element).on('evento', callback)	Coloca una escucha a un evento en un elemento.
	element.removeEventListener('evento', callback)	\$(element).off('evento', callback)	Quita una escucha al evento de un elemento.
Acciones Generales	setTimeout(function, tiempo)	\$.setTimeout(function, tiempo)	Establece el tiempo de retardo para la ejecución de una función.
Acciones para agregar y eliminar	element.appendChild(content)	\$(element).append(content)	Agrega contenido dentro del objeto, al final
	element.insertBefore(content, element.firstChild)	\$(element).prepend(content)	Agrega contenido dentro del objeto, al inicio
	element.parentNode.insertBefore(newElement, element.nextSibling)	\$(element).after(newElement)	Agrega contenido antes del objeto seleccionado
	element.parentNode.insertBefore(newElement, element)	\$(element).before(newElement)	Agrega contenido después del objeto seleccionado
	element.parentNode.removeChild(element)	\$(element).remove()	Elimina el elemento seleccionado y

			todo el contenido dentro de su estructura (hijos)
	<pre>while (element.firstChild) { element.removeChild(element.firstChild); }</pre>	<code>\$(element).empty()</code>	Elimina todo el contenido dentro de su estructura (hijos)
	<pre>Let cloned = element.cloneNode(true)</pre>	<pre>Let cloned = \$(element).clone()</pre>	Crea una copia exacta del objeto seleccionado
Acciones de efectos	<code>element.style.display = "none"</code>	<code>\$(element).hide();</code>	Ocultar el objeto cambiando el "display" CSS. 3 parámetros opcionales: velocidad, curvatura y callback
	<code>element.style.display = ""</code>	<code>\$(element).show();</code>	Mostrar/ Esconder el objeto cambiando el "display" CSS. 3 parámetros opcionales: velocidad, curvatura y callback
	<pre>if (window.getComputedStyle(element).display === "none") { element.style.display = ""; } else { element.style.display = "none"; }</pre>	<code>\$(element).toggle();</code>	
	<code>element.style.opacity = "1";</code>	<code>\$(element).fadeIn();</code>	Efecto fundido del objeto, cambiando la opacidad de 0 a 100% y el display. 2 parámetros opcionales: velocidad y callback
	<code>element.style.opacity = "0";</code>	<code>\$(element).fadeOut();</code>	
	<pre>if (window.getComputedStyle(element).opacity === "0") { element.style.opacity = "1"; } else { element.style.opacity = "0"; }</pre>	<code>\$(element).fadeToggle();</code>	

	<pre>element.style.transition = "opacity " + duration + "ms"; element.style.opacity = opacity;</pre>	<pre>\$ (element).fadeTo(duration, opacity);</pre>	Derivada de la anterior, permite variar la opacidad a un porcentaje de inicio. Como segundo parámetro se agrega el porcentaje en decimales
	<pre>element.style.transition = "height 0.3s"; element.style.height = "0";</pre>	<pre>\$(element).slideUp();</pre>	Función similar a la anterior, pero con efecto perciana de arriba hacia abajo, también alterando el display
	<pre>element.style.transition = "height 0.3s"; element.style.height = "auto";</pre>	<pre>\$(element).slideDown();</pre>	
	<pre>if (window.getComputedStyle(element).height === "0px") { element.style.transition = "height 0.3s"; element.style.height = "auto"; } else { element.style.transition = "height 0.3s"; element.style.height = "0"; }</pre>	<pre>\$(element).slideToggle();</pre>	
	<pre>element.style.transition = "all " + duration + "ms " + easing; Object.assign(element.style, properties);</pre>	<pre>\$ (element).animate(properties, duration, easing, complete);</pre>	Acepta varios parámetros CSS como matriz, para animar los objetos alterando posición, opacidad, tamaño, etc.
	<pre>element.style.transition = "none";</pre>	<pre>\$(element).stop();</pre>	Detiene la animación seleccionada o

			todas.
Acciones para navegar el DOM	element.parentElement	\$(element).parent()	Selecciona el padre o contenedor del objeto
	<pre>let ancestors = []; let current = element.parentElement; while (current) { if (!selector current.matches(selector)) { ancestors.push(current); } current = current.parentElement; }</pre>	\$(element).parents(selector)	Seleccionar cualquier padre del objeto referenciado
	Array.from(element.children).filter(child => child.matches(selector))	\$(element).children(selector)	Selecciona el elemento a continuación en la jerarquía
	Array.from(element.querySelectorAll(selector))	\$(element).find(selector)	Busca objeto descendente desde la referencia
	Array.from(element.parentElement.children).filter(child => child !== element && child.matches(selector))	\$(element).siblings(selector)	Selecciona los hermanos del objeto referencia
	element.nextElementSibling	\$(element).next(selector)	Hermano siguiente o anterior.
	element.previousElementSibling	\$(element).prev(selector)	
	element.parentElement.firstChild	\$(element).first()	Primero o último de los hijos anidados
	element.parentElement.lastElementChild	\$(element).last()	



Importante:

- Algunos métodos de manipulación de contenido, como ``html()`` y ``text()``, pueden funcionar de manera similar, pero tienen diferencias sutiles.
- La selección de elementos en jQuery a menudo se realiza utilizando el signo ``$`` seguido de paréntesis y el selector CSS, mientras que en JavaScript puro se utilizan métodos específicos del DOM.
- jQuery a menudo proporciona métodos más cortos y legibles para realizar tareas comunes, pero también agrega una capa de abstracción que puede tener un pequeño costo de rendimiento.

Funciones

La creación y el uso de funciones en jQuery no difieren significativamente de JavaScript puro. jQuery proporciona algunas funciones adicionales que pueden encadenarse para mejorar la legibilidad del código, pero la sintaxis básica y el propósito de las funciones son consistentes entre ambos.

Las equivalencias de las funciones, comparados con JavaScript, son las siguientes:

Tabla 4

Equivalencias de las funciones entre JS y jQuery

Funciones		
Tipo	JavaScript	jQuery
Funciones nombradas	<pre>function miFuncion(parametro) { /* Código */ }</pre>	<pre>function miFuncion(parametro) { /* Código */ }</pre>
Funciones anónimas	<pre>function() { /* Código */ }();</pre>	<pre>(function() { /* Código */ })();</pre>

Bucles

jQuery en sí mismo no proporciona bucles específicos. En cambio, utiliza la funcionalidad de bucles proporcionada por JavaScript., como for, forEach y while. Sin embargo, jQuery tiene la función each, que se utiliza para iterar sobre un conjunto de elementos, como un array o un objeto, y ejecutar una función para cada elemento sin tener que utilizar bucles tradicionales.

Figura 6

La sintaxis básica de each

```
$.each(coleccion, function(index, value) {  
    // Código a ejecutar para cada elemento  
});
```

Después de each se define colección, que contiene la colección de elementos sobre la cual iterar (puede ser un array, un objeto, u otro conjunto de elementos).Luego se define la función a ejecutarse por cada elemento de la colección. Ésta recibe dos parámetros: index, que es el índice del elemento actual en la colección, y value que es el valor del elemento.

Figura 7

Ejemplo de iteración sobre un array de números

```
let numeros = [1, 2, 3, 4, 5];  
  
$.each(numeros, function(index, value) {  
    console.log("Elemento en el índice " + index + ": " + value);  
});
```

Figura 8

Ejemplo de iteración sobre un objeto

```
let persona = {  
  nombre: "Juan",  
  edad: 30,  
  ciudad: "Ejemploville"  
};  
  
$.each(persona, function(key, value) {  
  console.log("Propiedad " + key + ": " + value);  
});
```

De JavaScript a jQuery

Modificar un código de JavaScript a jQuery generalmente implica reemplazar ciertas sintaxis y estructuras de código con las funciones y métodos proporcionados por jQuery. A continuación, encontrarás una pequeña guía para ayudarte en la transición.

1. Incluye jQuery en tu proyecto:

Asegúrate de incluir la biblioteca jQuery en tu HTML. Puedes descargar jQuery e incluirlo localmente o utilizar una CDN. Aquí hay un ejemplo de cómo incluir jQuery desde una CDN.

Figura 9

Script para enlazar jQuery desde jQuery.com

```
<script src="https://code.jquery.com/jquery-3.6.4.min.js"></script>
```

2. Cambia la selección de elementos:

Reemplaza las selecciones de elementos utilizando funciones nativas de JavaScript con las funciones de selección de jQuery.

Figura 10

Ejemplo de cambio de selección de elementos de JavaScript a jQuery

```
// JavaScript puro  
var elemento = document.getElementById('miElemento');  
  
// jQuery  
var elemento = $('#miElemento');
```


3. Modifica el manejo de eventos:

Cambia el manejo de eventos utilizando las funciones de eventos de jQuery.

Figura 11

Ejemplo de cambio de manejo de eventos de JavaScript a jQuery

```
// JavaScript puro
document.getElementById('miBoton').addEventListener('click', function() {
    alert('Presionaste el botón');
});

// jQuery
$('#miBoton').on('click', function() {
    alert('Presionaste el botón');
});
```

4. Utiliza las funciones de animación y manipulación de DOM de jQuery:

Reemplaza las operaciones de animación y manipulación de DOM utilizando las funciones proporcionadas por jQuery.

Figura 12

Ejemplo de cambio de manipulación del DOM de JavaScript a jQuery

```
// JavaScript puro
var elemento = document.getElementById('miElemento');
elemento.style.color = 'red';

// jQuery
$('#miElemento').css('color', 'red');
```

5. Adapta cualquier otra lógica específica de JavaScript:

Revisa cualquier lógica específica de JavaScript y ajusta el código según las funciones equivalentes de jQuery. Por ejemplo, si tienes un bucle for en JavaScript puro, podrías reescribirlo usando la función each de jQuery.

Figura 13

Ejemplo de cambio de lógicas de JavaScript a jQuery

```
// JavaScript puro
var elementos = document.querySelectorAll('.miClase');
for (var i = 0; i < elementos.length; i++) {
    // Lógica para cada elemento
}

// jQuery
$('.miClase').each(function() {
    // Lógica para cada elemento
});
```

Ejemplo de conversión de una página web que utiliza JavaScript a una que utiliza jQuery

Para comprender con claridad cómo jQuery puede simplificar la sintaxis y estructura del código, revisemos el siguiente ejemplo de conversión, reemplazando las funciones y métodos propios de JavaScript con los proporcionados por jQuery. Revisemos las figuras 14 y 15 que muestran dos versiones de un código HTML que realiza la misma función, añadir interactividad a una página web, pero con dos enfoques diferentes: JavaScript puro y jQuery.

En la figura 14 se utiliza JavaScript para agregar un evento al botón con id="miBoton". Este botón tiene con un atributo onclick que llama a la función presionarBoton() cuando el usuario hace clic en él. Esta función, definida en una etiqueta <script> dentro del cuerpo del documento, muestra un mensaje de alerta y cambia el texto del botón al que se le hizo clic. En este caso, a "Botón Presionado".

Además, se define explícitamente un manejador de eventos en el atributo de un elemento HTML y se manipulan los elementos del DOM directamente a través de métodos como getElementById y propiedades como innerHTML.

Figura 14

Página hecha con JavaScript

```
<!DOCTYPE html>
<html lang="es">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejemplo con JavaScript</title>
</head>

<body>

  <button id="miBoton" onclick="presionarBoton()">Presióname</button>

  <script>
    function presionarBoton() {
      alert('Has presionado el botón');

      // Cambiar el texto del botón
      let boton = document.getElementById('miBoton');
      boton.innerHTML = 'Botón Presionado';
    }
  </script>

</body>

</html>
```

En tanto a la página hecha con jQuery de la figura 15, vemos que se implementa la misma funcionalidad: al hacer clic en el botón, se muestra una alerta y se cambia el texto del botón. Sin embargo, en lugar de usar atributos de eventos como onclick directamente en las etiquetas HTML para ejecutar funciones JavaScript, se utiliza el método .click() de jQuery para adjuntar un manejador de eventos.

Por otro lado, el uso de `$(document).ready()` garantiza que el código de jQuery no se ejecute hasta que el DOM esté completamente cargado y las operaciones con el DOM, como cambiar el texto de un elemento, son más concisas con métodos como `.text()`.

Figura 15

Página hecha con jQuery

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Ejemplo con jQuery</title>
  <script src="https://code.jquery.com/jquery-3.6.4.min.js"></script>
</head>
<body>

  <button id="miBoton">Presióname</button>

  <script>
$(document).ready(function() {
  $('#miBoton').click(function() {
    alert('Has presionado el botón');

    // Cambiar el texto del botón
    $(this).text('Botón Presionado');
  });
});
</script>

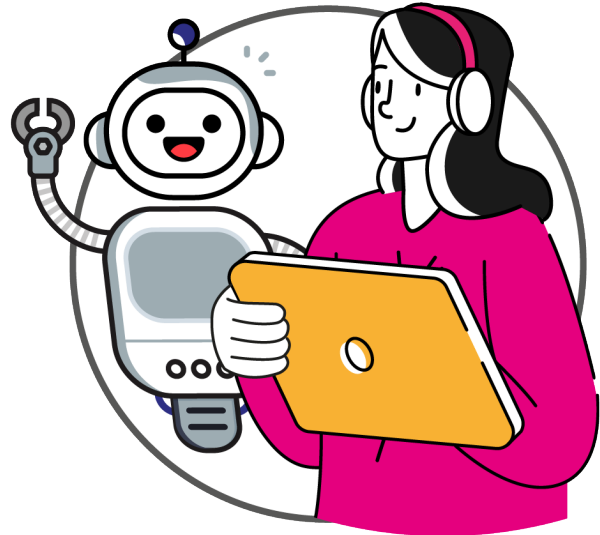
</body>
</html>
```

Al comparar estas versiones de código, vemos que ambas activan una alerta y actualizan el texto de un botón cuando este es presionado. Sin embargo, la versión con jQuery es más simple, limpia y modular. Además, favorece las buenas prácticas de programación al evitar la mezcla de JavaScript con las etiquetas HTML.

Cierre de la semana

Esta semana aprendiste que jQuery es una biblioteca de JavaScript que simplifica la manipulación del DOM y el manejo de eventos, así como la realización de animaciones en las páginas web. Optimiza la escritura de código ya que requiere menos líneas de código para realizar las mismas tareas que en JavaScript puro.

También descubriste las equivalencias entre las funciones de JavaScript y jQuery, lo que permite trabajar con un código sea más conciso y fácil de leer. Por ejemplo, que `document.getElementById('id')` en JavaScript puro equivale a `$('#id')` en jQuery. Además, aprendiste cómo migrar tus aplicaciones de JavaScript nativo a la biblioteca jQuery. De esta forma, ya puedes optimizar el código de tus páginas, de forma de hacer más fácil la programación.



Referencias

- Ramírez, L. (Comp.). (2023) *JavaScript / 5 cosas que hay que saber*
<http://js.luisfel.cl>
- Mozilla Web Docs, Avanzado, Chile (s.f.). *Guía de JavaScript*
<https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Introducci%C3%B3n>

[illegible]



DuocUC[®] ONLINE

Facilitador disciplinar: Claudio Navarro

Asesora par: Paola Véliz

Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.